CSCE 3701 · SOFTWARE ENGINEERING · FALL 2026

Closet

Milestone 1 — Requirements Specification and Architectural Design

Team	Habiba (requirements & user research) · Salma (frontend) · Bashar (recommendation engine & AI; use case diagram) · Karim (backend & database) · Youssef (testing, integration & deployment; component diagram)
Product owners	Course instructor and TA
Date / version	October 13, 2026 · Version 1.0 (living document, updated every sprint)

> Closet is a mobile, AI-assisted digital wardrobe for CSCE 3701 Software Engineering (Fall 2026), built by Habiba, Salma, Bashar, Karim and Youssef. This document is our requirements specification and our architectural design with its description: 70 user stories in 10 epics, 14 use cases, 13 measurable non-functional requirements, and a modular-monolith architecture, with 50 stories (all 43 Must) committed for December 1, 2026. It is a living document, updated at every sprint review.

Contents

1 System overview

2 Response to Milestone 0 feedback

3 Requirements specification
- 3.1 Domain vocabulary
- 3.2 Product backlog summary
- 3.3 Functional requirements (user stories)
- 3.4 Use case model
- 3.5 Non-functional requirements

4 Architectural design
- 4.1 Architectural style: a layered modular monolith
- 4.2 UML component diagram
- 4.3 Components and responsibilities
- 4.4 Interfaces and events
- 4.5 Data stores
- 4.6 Key runtime flows
- 4.7 AI design
- 4.8 Design patterns
- 4.9 Quality attributes and how the design achieves them
- 4.10 Technology stack and feasibility

5 Implementation scope and plan

6 Risks

7 Global, cultural, social, environmental, economic, public health, safety and welfare considerations

8 Traceability

9 Conclusion

How this document is organised

As requested for Milestone 1, this submission contains three parts: the requirements specification, the architectural design, and a description of that design. The table shows where each Milestone 1 item is.

Required item	Section
System overview: purpose, users, stakeholders, motivation	1
Response to Milestone 0 feedback	2
Requirements specification: product backlog, user stories, acceptance criteria	3.1 – 3.3
UML use case diagram and use case descriptions	3.4
Non-functional requirements	3.5
Architectural design: UML component diagram	4.1 – 4.2
Description of the design: components, responsibilities, relationships, interfaces	4.3 – 4.5
Runtime flows, AI design, design patterns, quality attributes, technology and feasibility	4.6 – 4.10

Required item	Section
Implementation scope (what will be fully built by Dec 1)	5
Risks	6
Global, cultural, social, environmental, economic, public health, safety and welfare considerations	7
Traceability and conclusion	8 – 9

Team and roles. Habiba – requirements and user research · Salma – frontend · Bashar – recommendation engine and AI, UML use case diagram · Karim – backend and database · Youssef – testing, integration and deployment, UML component diagram. Product owners: the instructor and TA.

Living document. At every sprint review we update story status, record new decisions, and check that the backlog, diagrams and code still match.

Version	Date	Change
0.1	Sep 2026	Milestone 0: problem, users, first feature list
1.0	Oct 13, 2026	Milestone 1 baseline: requirements and architecture
1.1	Oct 24, 2026 (planned)	Sprint 1 review update

1 System overview

The problem. People lose time every morning deciding what to wear, wear a small part of their wardrobe again and again, forget clothes they own, and buy things they do not need, which costs money and adds to textile waste.

The product. Closet is a mobile app that knows which clothes a user owns and which are in the laundry. The user picks an occasion and a style; Closet returns three complete outfits of four pieces that fit the weather, each with the reasons it works. The user can change one piece, try the outfit on a stylised avatar, wear it, save it and rate it. An AI chat, Ask Closet, answers questions and acts on the closet. Closet learns from every reaction and shows what it learned in plain words, with Undo.

The daily loop: recommend → react (change a piece, reject with a reason) → try on → wear and rate → learn → recommend better.

Target users	What they need
University students (primary)	Fast daily outfits, laundry-aware, buy less
Young professionals	Correct dress codes for work and interviews; planning for events
People with large wardrobes	Remember and use what they own
Users who need modest options	Covered shoulders and longer hemlines, respected everywhere

Stakeholders: end users; our team; the instructor and TA as product owners; the providers of the language-model, vision and weather services.

In scope: closet by photo and details, availability and laundry, recommendations with explanations, try-on avatar, AI chat that acts, feedback and learning, outfit builder, planning for events, wear analytics, smart shopping list.

Out of scope for this release: photo-real try-on on the user's own photo, packing assistant, Arabic interface, bulk upload, social sharing, any purchasing or payment.

2 Response to Milestone 0 feedback

Each of the five comments changed the product.

Feedback	What we changed	Where
Use an AI prompt to ask, modify and suggest changes	Ask Closet chat that acts through Closet functions (“change the shoes”, “put my jeans in the laundry”), shows the function used, and offers Undo	UC-07, US-31, US-76
How will the wardrobe get into the app – pictures?	Every piece is a photo plus structured details; optional AI tag suggestions that the user confirms before saving	UC-03, US-08 – US-11
Enrich and document the features, e.g. formal, business casual	One shared vocabulary: 5 dress-code levels, 9 occasions with default dress codes, 9 styles, 5 statuses, 7 rejection reasons	3.1
Use AI in the backend, the process and the frontend	AI in all three, with simulated parts of the prototype clearly labelled	3.5, 4.7
A camera inside the closet	Not practical to build or use reliably now; replaced by 5 availability statuses; a camera client stays a stretch idea	UC-06, US-56

3 Requirements specification

3.1 Domain vocabulary

The client, the recommendation engine, the AI chat and the database all use these exact values, so “business casual” means the same thing everywhere.

Dress-code level	1 Casual	2 Smart Casual	3 Business Casual	4 Semi-Formal	5 Formal
Default for occasions	Everyday, University, Travel	Dinner, Date night	Work, Interview	Party	Wedding

Term	Allowed values and rules
Styles	Casual, Chic, Minimal, Elegant, Classic, Sporty, Streetwear, Comfortable, Bold
Availability status	Available, Laundry, Borrowed, Packed, Unavailable – only Available pieces are ever used in an outfit
Rejection reasons (7)	Too formal · Too casual · Don’t like these colours · Not comfortable · Wore this recently · Don’t like one piece · Not in the mood today (hides the look for 7 days, no lasting change)
Pattern answers (after 2 rejections of a piece)	I don’t like it anymore (excluded) · Just not these days (paused 2 weeks) · Keep suggesting it (count reset)
Rating after wearing	1–5 stars or Loved it (raises similar colours and styles) · Wouldn’t repeat (combination retired)
Deletion reasons (4)	Became small → same item, new size · Torn or damaged → replacement · No longer my style → never suggested again · Donated, sold or other → no suggestion
Outfit	4 pieces: top or dress, bottom or layer, shoes, extra
Match criteria (builder)	Complete outfit · Colour harmony (≤ 2 accent colours, no clash) · Dress code (every main piece suits the level) · Weather fit (≥ 26 °C no warm pieces; < 19 °C a warm layer) · Style consistency (+ Modest when on)
Rule score	100 − 17 per failed criterion; always labelled “rule score”, never a fashion verdict
Utilization	Pieces worn in the last 30 days ÷ pieces owned
Forgotten piece	Worn at least once, but not in the last 40 days

3.2 Product backlog summary

70 user stories in 10 epics, prioritised with MoSCoW. Every Must story is committed for December 1.

Epic	Stories	Must	Should	Could	Committed	Stretch	Future	Main components
E1 Accounts & Profile	7	6	0	1	6	0	1	Accounts & Profile, Privacy & Settings
E2 Digital Closet	9	6	1	2	6	2	1	Wardrobe, Vision
E3 Availability & Laundry	5	3	1	1	4	0	1	Availability
E4 Outfit Recommender	9	7	2	0	8	1	0	Recommendation Engine, Outfits
E5 Virtual Try-On	4	0	1	3	1	2	1	Web App (avatar renderer)
E6 Ask Closet AI Agent	8	5	1	2	8	0	0	Ask Closet, Language Model Adapter
E7 Feedback & Learning	9	6	2	1	7	2	0	Recommendation Engine, Accounts & Profile

Epic	Stories	Must	Should	Could	Committed	Stretch	Future	Main components
E8 Outfit Builder	5	3	1	1	3	2	0	Outfits, Recommendation Engine
E9 Plan Ahead	7	3	1	3	3	3	1	Planner
E10 Saved & Wear Analytics	7	4	1	2	4	3	0	Outfits, Analytics
Total	**70**	**43**	**11**	**16**	**50**	**15**	**5**	

3.3 Functional requirements (user stories)

Each story reads “As a user, I want to …”. Release: S1–S3 = committed sprint, Stretch = only if time allows, Future = not in this course. Each acceptance criterion becomes a test.

E1 Accounts & Profile

ID	I want to…	Priority	Release	UC	Acceptance criteria
US-01	Register and log in with email or Google	Must	S1	UC-01	Password ≥ 8 chars or Google; wrong password shows an error; no other user’s data is ever returned
US-02	Answer a short style quiz: styles, colours, occasions	Must	S1	UC-02	≤ 4 steps, skippable; answers change the first recommendation
US-03	Set my usual dress code and a modest preference	Must	S1	UC-02	One of the 5 levels; with modest on, no non-modest piece appears anywhere
US-04	Set my city so outfits use my local weather	Must	S1	UC-02	Forecast on Home is for that city
US-05	Edit my profile and settings at any time	Must	S2	UC-02	Every quiz answer is editable; the next recommendation uses it
US-06	Delete my account and all my photos and data	Must	S1	UC-01	Double confirmation; all data and photos deleted within 24 h
US-07	Use the app in Arabic (RTL)	Could	Future	—	Out of scope

E2 Digital Closet

ID	I want to…	Priority	Release	UC	Acceptance criteria
US-08	Add a piece by taking or uploading a photo	Must	S1	UC-03	JPG/PNG/HEIC ≤ 10 MB stored and shown as the piece’s image
US-09	Enter category, colour, fabric, season, size, style, dress code	Must	S1	UC-03	Values from 3.1; category, colour and ≥ 1 dress-code level required
US-10	Let AI suggest those details from the photo	Should	Stretch	UC-03	Suggestions ≤ 10 s with confidence; nothing saved until confirmed; failure leaves a usable empty form
US-11	Review and edit every detail before saving	Must	S1	UC-03	One review screen; the piece exists only after Save
US-12	Browse and filter my closet by category and status	Must	S1	UC-04	Filters combine; counts update
US-71	Edit a piece’s details later	Must	S1	UC-04	Changes appear in all outfits and filters
US-72	Enter an optional purchase price	Could	Stretch	UC-04	Optional, ≥ 0, in EGP
US-13	Delete a piece with a reason	Must	S2	UC-05	Impossible without one of the 4 reasons; piece removed from saved and planned outfits
US-14	Upload several pieces at once	Could	Future	—	Out of scope

E3 Availability & Laundry

ID	I want to…	Priority	Release	UC	Acceptance criteria
US-56	Set a status per piece (5 statuses)	Must	S1	UC-06	Exactly one status, shown as a badge
US-15	Put a piece in the laundry basket and take it out	Must	S1	UC-06	One tap each way; basket lists all Laundry pieces
US-16	Return everything with one “Wash done” tap	Should	S1	UC-06	All Laundry pieces become Available
US-17	Never get an outfit with an unavailable piece	Must	S2	UC-09	Test: 100 generated outfits with random statuses contain 0 non-Available pieces
US-18	Reminder for pieces long in the laundry	Could	Future	—	Out of scope

E4 Outfit Recommender

ID	I want to…	Priority	Release	UC	Acceptance criteria
US-19	Choose an occasion and a style and get 3 outfits of 4 pieces	Must	S2	UC-09	3 outfits in ≤ 2 s; each complete; no two share more than 2 pieces when the closet allows
US-20	Have the weather shape the outfits	Must	S2	UC-09	≥ 26 °C: no warm piece; < 19 °C: a warm layer; ≥ 38 °C or ≤ 8 °C: heat or cold note
US-21	Filter by dress code and modest option	Must	S2	UC-09	Every main piece suits the level; modest respected
US-57	See why each outfit works	Must	S2	UC-09	3–5 reason lines and a “rule score” label on every outfit
US-23	Tap “Wear this” to log the outfit and its pieces	Must	S2	UC-12	Wear count +1 and last-worn = today for all 4 pieces; analytics update
US-24	Change one of the 4 pieces, the other 3 stay	Must	S2	UC-09	Only that slot changes; alternatives are Available; swap stored as feedback
US-26	Save an outfit, and mark it as a favorite	Must	S2	UC-09	Saved list; favorite = a starred saved outfit
US-59	“Style this piece”: 3 outfits with one piece fixed	Should	S2	UC-08	All 3 outfits contain that piece
US-58	Choose repeats: don’t mind / avoid last 7 days / something different	Should	Stretch	UC-09	“Avoid 7 days” uses no piece worn in the last 7 days when an alternative exists

E5 Virtual Try-On

ID	I want to…	Priority	Release	UC	Acceptance criteria
US-27	See an outfit on a stylised avatar	Should	S3	UC-09	All 4 pieces drawn in their colours in ≤ 1 s on the phone; no photo of the user
US-28	Flip between the 3 looks in the try-on	Could	Stretch	UC-09	Swipe switches looks
US-29	Choose the avatar’s skin tone	Could	Stretch	UC-09	≥ 6 tones, saved in profile
US-30	Try on with a photo of myself	Could	Future	—	Out of scope

E6 Ask Closet AI Agent

ID	I want to…	Priority	Release	UC	Acceptance criteria
US-31	Chat with an agent that knows my closet, calendar and weather	Must	S3	UC-07	Reply ≤ 8 s (p90); never names a piece the user does not own
US-76	Have the agent act for me through Closet functions	Must	S3	UC-07	Only whitelisted functions; reply lists the function used; every change undoable

ID	I want to…	Priority	Release	UC	Acceptance criteria
US-32	Ask “what does my closet need?”	Must	S3	UC-14	Item types listed, each with a reason
US-33	See torn or too-small pieces suggested for replacement	Must	S3	UC-14	Every Torn or Became-small deletion appears on the list
US-34	Never be told to re-buy “no longer my style” pieces	Must	S3	UC-14	No suggestion matches their category and colour
US-36	Keep a shopping list	Should	S3	UC-14	Add from chat or list, remove, persists
US-64	See closet gaps by category and dress code	Could	S1	UC-14	Gap = ≤ 1 piece at a level the user dresses for
US-65	Get a duplicate-purchase warning	Could	S3	UC-14	Same category and colour as an owned piece shows that piece

E7 Feedback & Learning

ID	I want to…	Priority	Release	UC	Acceptance criteria
US-37	Have wears, swaps and rejections stored as feedback	Must	S2	UC-09	Each event stored with time, outfit and pieces
US-60	Give a reason when I reject a look	Must	S2	UC-09	7 reasons; each changes exactly one preference
US-61	Hide a piece for 7 days without it being a dislike	Should	S1	UC-09	Returns after 7 days; rejection count unchanged
US-62	Rate an outfit after wearing it	Must	S3	UC-12	Wouldn’t repeat = combination never recommended again
US-38	Be asked when I reject the same piece twice	Must	S3	UC-09	One in-app question at the 2nd rejection, within the daily limit
US-39	Answer: don’t like it / not these days / keep suggesting	Must	S3	UC-09	Exclude, pause 2 weeks, or reset
US-40	See what Closet learned in plain words, and undo it	Must	S3	UC-02	One sentence per lesson; Undo restores the previous preference exactly
US-63	Have favorites raise similar colours and styles	Could	Stretch	UC-09	Favoriting raises their weights
US-75	Pause learning, reset it, or download my data	Should	Stretch	UC-02	Pause stops updates; download gives a JSON file

E8 Outfit Builder

ID	I want to…	Priority	Release	UC	Acceptance criteria
US-42	Pick a top/dress, bottom/layer, shoes and an extra	Must	S2	UC-11	Only Available pieces per slot
US-43	See a rule score with pass/fail for the 5 criteria	Must	S2	UC-11	Criteria and formula as in 3.1
US-66	Read one plain sentence that explains a mismatch	Must	S2	UC-11	Names the failing criterion
US-44	Get one suggested swap that improves the score	Could	Stretch	UC-11	Applying it raises the score
US-45	Try on, save or wear a built outfit	Should	Stretch	UC-11	Same actions as a recommended outfit

E9 Plan Ahead

ID	I want to…	Priority	Release	UC	Acceptance criteria
US-46	Add events by hand and see them in a calendar	Must	S1	UC-10	Title, date, time and one occasion; month view

ID	I want to…	Priority	Release	UC	Acceptance criteria
US-47	Get 3 outfits for an event	Must	S3	UC-10	Uses the event’s dress code, that day’s forecast and Available pieces
US-48	Choose one as the planned outfit	Must	S3	UC-10	Shown on the event and on Home that day
US-50	Sync events from Google Calendar	Should	Stretch	UC-10	Read-only; next 30 days
US-67	Get a reminder the night before	Could	Stretch	UC-10	One in-app notice at 21:00, within the daily limit
US-79	Be warned if a planned piece becomes unavailable	Could	Stretch	UC-10	Warning with a one-tap swap
US-68	Packing assistant for trips	Could	Future	—	Out of scope

E10 Saved & Wear Analytics

ID	I want to…	Priority	Release	UC	Acceptance criteria
US-51	See saved outfits and filter favorites	Must	S2	UC-09	A saved outfit with a non-Available piece shows a warning and cannot be worn
US-52	See most and least worn pieces	Must	S3	UC-13	Top and bottom 5 by wear count
US-69	See my wardrobe utilization	Must	S3	UC-13	Formula in 3.1; matches a manual count in tests
US-54	See forgotten pieces	Must	S3	UC-13	Exactly the pieces matching 3.1; each has “Style it”
US-53	See cost per wear for priced pieces	Could	Stretch	UC-13	Only priced pieces, with a note
US-70	Get a weekly forgotten-item challenge	Could	Stretch	UC-13	One per week, within the daily limit
US-74	Control notification types and the daily limit	Should	Stretch	UC-02	Types can be switched off; default 3 per day

3.4 Use case model

One primary actor uses 14 use cases, supported by four external services. «include» marks a step that always runs; «extend» marks optional behaviour.

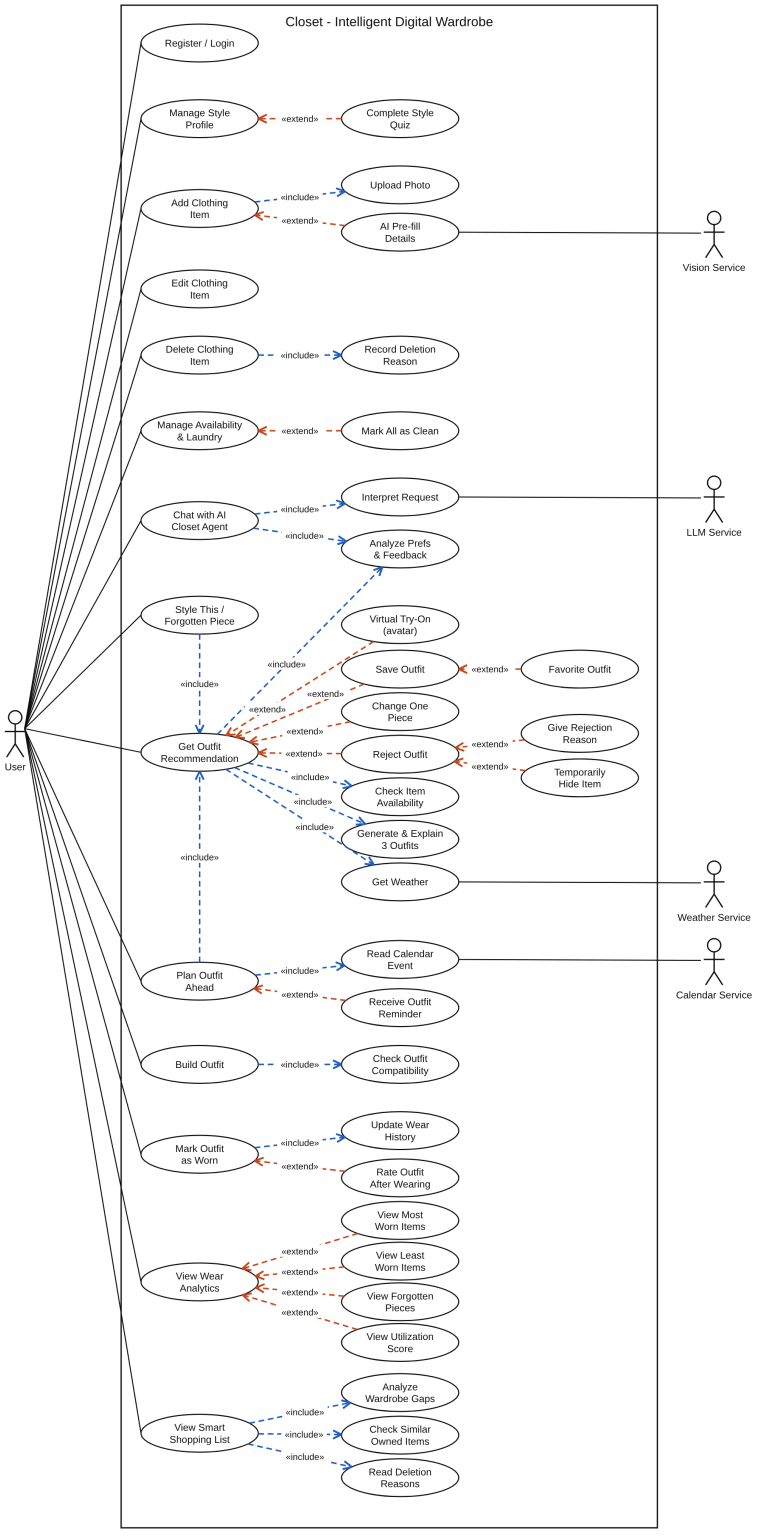


Figure 1 – UML use case diagram (prepared by Bashar). The avatar try-on is drawn inside the app and reminders are in-app notices, so neither has an external actor.

Actor	Type	Role
User	Primary	Owns a closet; starts every use case
Vision Service	External	Suggests a piece's tags from its photo (UC-03 extension)
LLM Service	External	Interprets chat messages into Closet functions (UC-07)
Weather Service	External	Current temperature and forecast for the user's city (UC-09)
Calendar Service	External	Upcoming events (UC-10); manual events work without it

Use case descriptions

Use case	Main flow	Relationships	Alternatives and exceptions	Stories
UC-01 Register / Login	Enter email + password or Google → account opened; first login starts the quiz	—	Wrong password → error; 5 failures lock 15 min; delete account from Settings	US-01, US-06
UC-02 Manage Style Profile	Edit styles, colours, occasions, dress code, modest, city; view and undo lessons	«extend» Complete Style Quiz (first login)	Quiz skipped → defaults	US-02–05, US-40
UC-03 Add Clothing Item	Take or pick photo → enter details → review → Save → piece Available	«include» Upload Photo; «extend» AI Pre-fill Details	Vision down or > 10 s → manual entry; missing field → Save disabled	US-08, 09, 11
UC-04 Edit Clothing Item	Browse and filter → open piece → edit details	—	From the piece: Style this piece, change status, delete	US-12, US-71
UC-05 Delete Clothing Item	Remove → choose 1 of 4 reasons → piece removed, reason kept	«include» Record Deletion Reason	Piece in a saved or planned outfit → user is warned, then it is removed from those outfits	US-13
UC-06 Manage Availability & Laundry	Set one of 5 statuses; laundry basket lists Laundry pieces	«extend» Mark All as Clean	—	US-15, US-16, US-56
UC-07 Chat with AI Closet Agent	Message → LLM picks a whitelisted function → run on the user's data → reply lists the function, offers Undo	«include» Interpret Request; «include» Analyze Prefs & Feedback	No match → one clarifying question; LLM down or > 8 s → keyword parser	US-31, US-76
UC-08 Style This / Forgotten Piece	Pick a piece → 3 outfits that all contain it	«include» UC-09	Piece not Available → offer to change status	US-59
UC-09 Get Outfit Recommendation	Detailed below	4 «include», 4 «extend»	Detailed below	US-17, 19–21, 24, 26, 27, 37, 38, 39, 51, 57, 60, 61
UC-10 Plan Outfit Ahead	Add or pick an event (manual, or synced as a stretch goal) → 3 outfits for its dress code and forecast → choose one	«include» Read Calendar Event; «include» UC-09; «extend» Receive Outfit Reminder	No calendar connected → manual events	US-46–48
UC-11 Build Outfit	Pick 4 pieces → pass/fail per criterion, rule score, one-sentence mismatch	«include» Check Outfit Compatibility	Suggested swap (stretch)	US-42, US-43, US-66
UC-12 Mark Outfit as Worn	Wear this → wear count and last-worn updated for all 4 pieces	«include» Update Wear History; «extend» Rate Outfit After Wearing	Undo in the same session	US-23, US-62

Use case	Main flow	Relationships	Alternatives and exceptions	Stories
UC-13 View Wear Analytics	Utilization score for the last 30 days	«extend» Most Worn, Least Worn, Forgotten Pieces, Utilization Score	No history yet → explanation	US-52, US-54, US-69
UC-14 View Smart Shopping List	Deletion reasons → replacements; gaps by category; duplicates flagged; list shown with a reason per item	«include» Analyze Wardrobe Gaps; «include» Check Similar Owned Items; «include» Read Deletion Reasons	Nothing needed → says so	US-32–34, US-36, US-64, US-65

UC-09 Get Outfit Recommendation (detailed)

Field	Description
Actors	User (primary), Weather Service
Precondition	Logged in; at least one Available top or dress, bottom and shoes
Main flow	1) User picks occasion and style (dress code, modest, repeats under More options). 2) «include» Get Weather. 3) «include» Check Item Availability: only Available pieces that are not disliked, paused or hidden. 4) «include» Analyze Prefs & Feedback. 5) «include» Generate & Explain 3 Outfits: 3 different outfits, each with a rule score and 3–5 reasons.
Extensions	Virtual Try-On (avatar) · Save Outfit, extended by Favorite Outfit · Change One Piece (only that slot changes) · Reject Outfit, extended by Give Rejection Reason and Temporarily Hide Item
Alternatives	Too few pieces for the dress code → relax one slot and say so · Weather down → last known or manual temperature · Same piece rejected twice → pattern question · Wear this → UC-12
Postcondition	3 explained outfits shown; every reaction stored as feedback

3.5 Non-functional requirements

Every quality requirement has a measurable target and a way to verify it.

ID	Attribute	Requirement	Verification
NFR-01	Performance	3 outfits in ≤ 2 s (95th percentile, closet ≤ 300 pieces); chat reply ≤ 8 s (90th percentile); avatar drawn in ≤ 1 s on the phone	Load test with a 300-piece closet
NFR-02	AI suggestions	Tag suggestions optional, ≤ 10 s; nothing saved without user confirmation	Test with the vision service slow and down
NFR-03	Usability	A new user finishes the quiz and adds a first piece in < 3 min; core tasks ≤ 3 taps	Test with 5 users
NFR-04	Security	HTTPS only; hashed passwords; login tokens checked at the gateway; every query scoped to the user; lockout after 5 failed logins	Test that user A cannot read user B's data
NFR-05	Privacy	Designed with Egypt's PDPL (Law 151/2020) principles in mind: consent, minimal data, export, deletion within 24 h; only message text and garment photos (no location data, no faces) leave our servers	Privacy checklist each sprint; deletion test
NFR-06	Reliability	If the language model, weather or vision service fails, the app keeps working: keyword parser, manual weather, manual tags; never a crash	Tests with each adapter forced to fail
NFR-07	Accessibility	WCAG 2.1 AA target: contrast ≥ 4.5:1, touch targets ≥ 44 px, screen-reader labels, never colour alone	Automated audit + manual check
NFR-08	Portability	Works on current Chrome (Android) and Safari (iOS), from 360 px wide	Test on 4 devices or emulators
NFR-09	Testability	≥ 60% unit-test coverage of core modules; every Must acceptance criterion has a test; tests run on every pull request	Coverage report

ID	Attribute	Requirement	Verification
NFR-10	Explainability	Every outfit shows its reasons; every score labelled “rule score”; every lesson listed in plain words	UI review
NFR-11	Localisation readiness	All text in one resource file; layout supports right-to-left; °C and EGP	Code review
NFR-12	Cost	AI calls cached and capped per user per day (gateway AI quota); within free or student tiers	Weekly usage check
NFR-13	Notifications	At most 3 in-app notices per user per day	Unit test

Data requirements. Each piece stores its owner, photo, category, colour, fabric, season, size, styles, dress-code levels, modest flag, status, wear count and last-worn date. Feedback events (wear, swap, rejection with reason, rating, hide) are the only input to learning. Deletion reasons are kept after the piece is deleted, without the photo. User data is never used to train models or shared with other users.

Process requirements (AI in our team process). PR-01 AI may draft stories and acceptance criteria; the requirements owner approves them. PR-02 AI coding assistants may be used; every change is reviewed by a teammate. PR-03 AI may draft tests from acceptance criteria. PR-04 AI review comments and automated tests run on every pull request; a human approves the merge. PR-05 AI may help update this document at each sprint review; the owner checks every change.

External interfaces. Users reach Closet through a phone browser (5 tabs: Home, Outfits, Ask, Closet, More). The backend talks to a hosted language-model API, a weather API (e.g. Open-Meteo) and a vision API over HTTPS, each through an adapter (section 4). The phone camera is used through the browser.

4 Architectural design

4.1 Architectural style: a layered modular monolith

Closet runs as one backend process split into modules with explicit interfaces, behind a single API gateway, with the client, services, adapters and data in separate tiers.

Option	Benefits	Costs for our team	Decision
Modular monolith	One codebase, one deployment, one database, fast in-process calls, simple debugging; module boundaries kept by interfaces	The whole backend scales as one unit	**Chosen**
Microservices	Independent scaling and deployment	Many deployables, message broker, service discovery, distributed tracing – far beyond 280 hours, no benefit to users at our scale	Rejected
Unstructured monolith	Fastest first sprint	Features tangle; the AI chat could reach into any table; hard to test or split later	Rejected

Because every module already sits behind an interface, any module (most likely Ask Closet, if AI traffic grows) can later become its own service by replacing in-process calls with HTTP and the in-process event bus with a broker.

4.2 UML component diagram

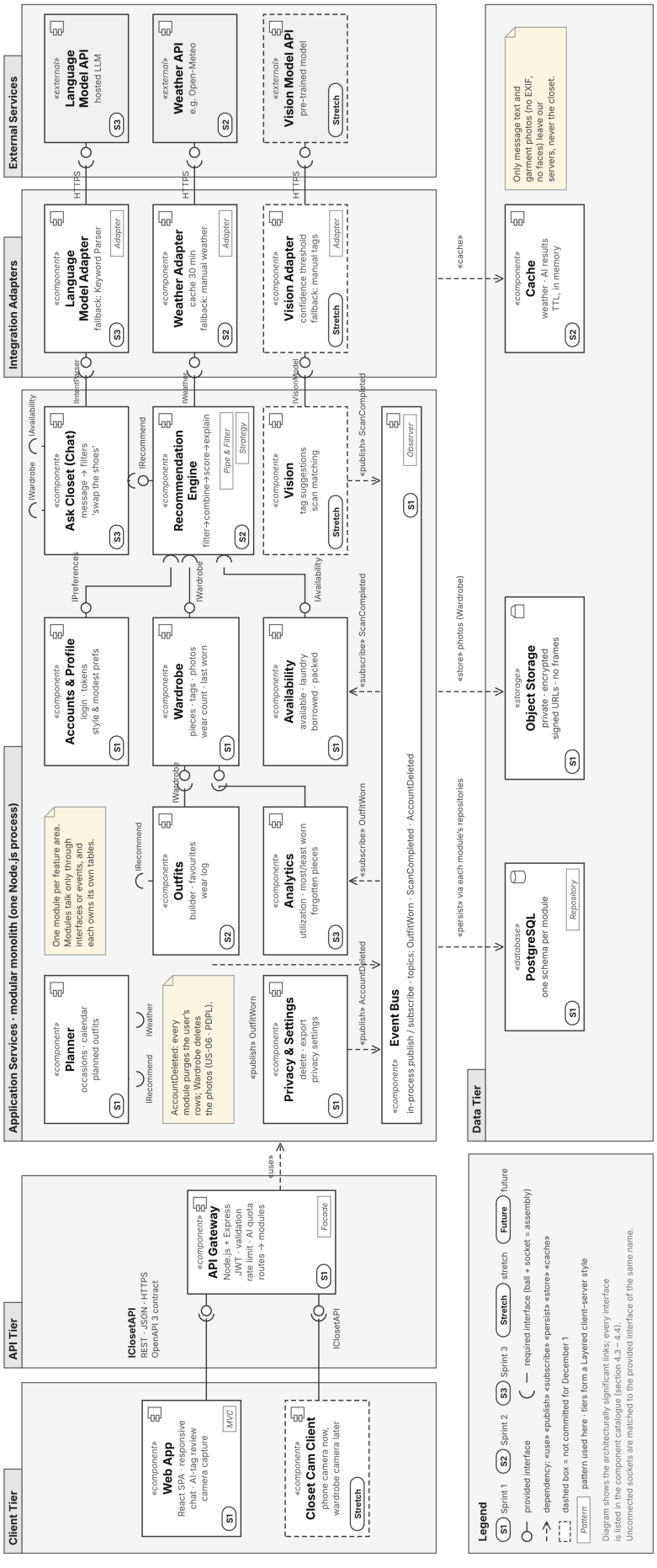


Figure 2 – UML component diagram (prepared by Youssef). Lollipops are provided interfaces, sockets are required interfaces (an unconnected socket is matched to the provided interface of the same name), dashed

arrows are dependencies («use», «publish», «subscribe», «persist», «store», «cache»). S1–S3 badges show the sprint; dashed boxes are not committed for December 1.

How to read it, left to right:

1. **Client tier** – the Web App (React, MVC) on the phone. A camera client is a stretch idea.
2. **API tier** – the API Gateway, the only entry point, provides IClosetAPI (REST, JSON, HTTPS, OpenAPI contract).
3. **Application services** – one Node.js process with one module per feature area. Modules talk only through interfaces or through the Event Bus, and each owns its own tables.
4. **Integration adapters** – the only code that calls external services; each has a fallback.
5. **External services** – hosted language model, weather and vision APIs.
6. **Data tier** (bottom) – PostgreSQL, object storage for photos, and a cache.

4.3 Components and responsibilities

Component	Responsibility	Provides	Requires / relationships	Use cases
Web App «MVC» (+ Closet Cam Client, stretch)	All screens, chat, AI-tag review, camera capture, avatar try-on drawn on the phone	UI	IClosetAPI	all
API Gateway «Facade»	Single entry point: login-token check, validation, rate limits, AI quota, routing to modules	IClosetAPI	all module interfaces	all
Accounts & Profile	Login, tokens, style and modest preferences, learned lessons	IPreferences	—	UC-01, UC-02
Privacy & Settings	Delete account, export data, privacy settings	—	publishes AccountDeleted	UC-01, UC-02
Wardrobe	Pieces, tags, photos, wear count, last worn, deletion reasons	IWardrobe	Object Storage	UC-03, UC-04, UC-05
Vision	Tag suggestions for new photos	—	IVisionModel; publishes ScanCompleted	UC-03
Availability	Statuses and laundry basket	IAvailability	subscribes ScanCompleted	UC-06
Recommendation Engine «Strategy» «Pipe & Filter»	Filter → combine → score → explain; change one piece; Style this piece; builder check; learning from feedback	IRecommend	IWardrobe, IAvailability, IPreferences, IWeather	UC-08, UC-09, UC-11
Ask Closet (Chat)	Turn a message into Closet functions; smart shopping list	—	IIntentParser, IRecommend, IWardrobe, IAvailability	UC-07, UC-14
Outfits	Builder, saved and favorite outfits, wear log, ratings	—	IWardrobe, IRecommend (builder check); publishes OutfitWorn	UC-05, UC-09, UC-11, UC-12

Component	Responsibility	Provides	Requires / relationships	Use cases
Analytics	Utilization, most and least worn, forgotten pieces	—	IWardrobe; subscribes OutfitWorn	UC-13, UC-14
Planner	Occasions, calendar events, planned outfits	—	reuses IRecommend + IWeather	UC-05, UC-10
Event Bus «Observer»	In-process publish/subscribe	publish / subscribe	—	—
Language Model Adapter «Adapter»	Calls the hosted LLM; fallback: keyword parser	IIntentParser	Language Model API	UC-07
Weather Adapter «Adapter»	Forecast by city, cached 30 min; fallback: manual weather	IWeather	Weather API	UC-09, UC-10
Vision Adapter «Adapter»	Confidence threshold; fallback: manual tags	IVisionModel	Vision Model API	UC-03

4.4 Interfaces and events

Interface / event	Offered by	Used by	Purpose
IClosetAPI	API Gateway	Web App	REST/JSON over HTTPS, one OpenAPI contract
IPreferences	Accounts & Profile	Recommendation Engine	Styles, colours, dress code, modest, learned weights
IWardrobe	Wardrobe	Recommendation Engine, Outfits, Analytics, Ask Closet	Read pieces and their details
IAvailability	Availability	Recommendation Engine	Only Available pieces are used
IRecommend	Recommendation Engine	Ask Closet, Planner	3 explained outfits, alternatives for one slot
IIntentParser	Language Model Adapter	Ask Closet	Message → whitelisted Closet function
IWeather	Weather Adapter	Recommendation Engine, Planner	Temperature and forecast
IVisionModel	Vision Adapter	Vision	Suggested tags with confidence
OutfitWorn	Outfits	Analytics (subscriber)	Update wear history and utilization
ScanCompleted	Vision	Availability (subscriber)	A scanned piece is matched and its status updated
AccountDeleted	Privacy & Settings	every module	Each module purges the user's rows; Wardrobe deletes the photos

Ask Closet has no tables of its own: IIntentParser picks a whitelisted function, and Ask Closet carries it out only by calling other modules' provided interfaces (IRecommend for outfits, IWardrobe and IAvailability for pieces, statuses and deletion reasons). This is how we guarantee the AI chat cannot touch the database directly.

4.5 Data stores

Store	Holds	Design
PostgreSQL	All structured data: users, profiles, pieces, outfits, wears, feedback, events, shopping list	One schema per module, accessed only through that module's repositories «Repository»; every query takes the user id

Store	Holds	Design
Object Storage	Piece photos	Private, encrypted, short-lived signed URLs, no frames
Cache	Weather and AI results	In memory with a time-to-live, to meet NFR-01 and NFR-12

4.6 Key runtime flows

Get outfit recommendations (UC-09).

1. Web App sends the occasion and style to the API Gateway through IClosetAPI.
2. The gateway checks the token and routes to the Recommendation Engine.
3. The engine gets the forecast through IWeather (cache first), Available pieces through IAvailability and IWardrobe, and preferences through IPreferences.
4. It filters, combines and scores the pieces, and returns 3 outfits with reasons.

Chat request with an action (UC-07), e.g. “change the shoes”.

1. Ask Closet sends the message to IIntentParser; the Language Model Adapter returns a whitelisted function, e.g. change one piece.
2. Ask Closet calls IRecommend for an Available alternative and applies it.
3. The reply names the function used and offers Undo. If the LLM is down, the keyword parser maps the same request.

Wear an outfit (UC-12). Outfits stores the wear and publishes OutfitWorn; Analytics receives it through the Event Bus and updates utilization, without Outfits knowing Analytics exists.

4.7 AI design

Where	What the AI does	Safeguard
Frontend	Ask Closet chat, AI-tag review, a reason on every outfit, avatar try-on	Simulated parts of the prototype are labelled on screen
Backend: recommendation	Rule-based scoring (occasion, style, colours, weather, learned weights) with explanations; learns from ratings, reasons and swaps	Every lesson shown in plain words with Undo; Strategy lets a learned scorer replace the rules later
Backend: language model	Turns a message into one of our whitelisted functions	Cannot invent pieces or write to the database; keyword-parser fallback; daily quota
Backend: vision	Suggests tags from a photo	Confidence threshold; the user confirms; manual tags fallback
Team process	Drafts stories, tests and reviews; helps update this document	A teammate reviews every result (PR-01 – PR-05)

We train no models; we use hosted services. In the Milestone 1 prototype the language model, vision and weather are simulated and labelled; the scoring and learning rules really run. Behind the adapters, the weather API is connected in Sprint 2 and the language-model API in Sprint 3; vision is a stretch goal.

4.8 Design patterns

Pattern	Where	Quality attribute it serves
Layered (client–server tiers)	Whole system	Maintainability
Facade	API Gateway	Security, a simple client
Strategy	Recommendation Engine scorer	Modifiability
Pipe and Filter	Recommendation Engine: filter → combine → score → explain	Modifiability, testability
Observer	Event Bus	Loose coupling, extensibility
Repository	PostgreSQL access per module	Testability, security

Pattern	Where	Quality attribute it serves
Adapter	Language Model, Weather and Vision adapters	Availability, ability to switch providers
MVC	Web App	Maintainability

4.9 Quality attributes and how the design achieves them

Quality attribute	Design decision	NFR
Performance	In-process calls; bounded search; weather and AI results cached; avatar drawn on the phone	NFR-01
Security	Single gateway checks every request; repositories always filter by user	NFR-04
Privacy	Private encrypted photo storage; minimal data sent to AI; AccountDeleted purge	NFR-05
Availability / reliability	Every external service behind an adapter with a fallback	NFR-06
Modifiability	Strategy scorer; adapters allow switching providers; modules can be split later	—
Testability	Interfaces and repositories can be mocked; tests on every pull request	NFR-09
Explainability	Reasons come from the same rules that score the outfit	NFR-10

4.10 Technology stack and feasibility

Layer	Technology
Client	React single-page app, responsive, installable on the phone
Backend	Node.js + Express, one process
Data	PostgreSQL; S3-compatible object storage; in-memory cache
AI and weather	Hosted language-model and vision APIs; Open-Meteo weather
Quality	Unit and end-to-end tests, automated on every pull request

Why it is feasible. The team already knows React, Node.js and PostgreSQL. We train no models. The core loop (scoring, filtering, explanations, learning) already runs in our clickable prototype. Every external service has a free tier and a fallback, so the app works even with all of them off. The 50 committed stories are sized at 271 hours against 280 hours of capacity (section 5).

5 Implementation scope and plan

We commit to fully building 50 of the 70 stories, including all 43 Must stories, by December 1, 2026. Every use case in Figure 1 has committed stories; stretch stories are optional extensions started only if a sprint finishes early.

Sprint	Dates	Goal	Committed stories	Effort
S1	Oct 11 – 24	A working closet	US-01, 02, 03, 04, 06, 08, 09, 11, 12, 71, 15, 16, 56, 61, 64, 46 (16)	75 h
S2	Oct 25 – Nov 7	Outfits that explain themselves	US-05, 13, 17, 19, 20, 21, 57, 23, 24, 26, 37, 42, 43, 66, 51, 59, 60 (17)	78 h
S3	Nov 8 – 21	AI chat, learning, planning, analytics	US-27, 31, 76, 32, 33, 34, 36, 65, 62, 38, 39, 40, 47, 48, 52, 69, 54 (17)	78 h
Release	Nov 22 – Dec 1	Integration, testing, demo	Real language-model adapter switched on, end-to-end tests of every Must criterion	40 h

Stretch (15): US-10 AI tag pre-fill · US-72 purchase price · US-58 repeat control · US-28 flip looks in try-on · US-29 avatar skin tones · US-63 favorites influence · US-75 pause/reset learning · US-44 builder swap · US-45 actions on built outfits · US-50 Google Calendar sync · US-67 night-before reminder · US-79 planned-piece re-check · US-53 cost per wear · US-70 forgotten challenge · US-74 notification settings.

Future (5): US-07 Arabic interface · US-14 bulk upload · US-18 laundry reminder · US-30 photo-real try-on · US-68 packing assistant.

Capacity check. Large stories (login, photo upload, recommender, chat, chat actions, avatar, builder score, events calendar, rejection reasons) are sized at 12 h each, all others at 3 h, including tests: 9 × 12 + 41 × 3 = 231 h, plus 40 h of integration = **271 h**. Capacity is 5 members × 8 h/week × 7 weeks = **280 h** (80 h per sprint). If a sprint falls behind, stretch stories are dropped first, then Could, never Must. New stories need product-owner approval and are added at sprint planning.

6 Risks

ID	Risk	Likelihood	Impact	Mitigation	Owner
R1	Language model slow, costly or unavailable	Medium	High	Whitelisted functions only; 8 s timeout; cache and daily quota; keyword-parser fallback	Bashar
R2	Learning draws the wrong conclusion	Medium	Medium	Reasons give clean signals; “not in the mood” is temporary; ask after 2 rejections; every lesson undoable	Bashar
R3	Avatar try-on looks unrealistic	Medium	Medium	Stylised avatar; the 4 pieces are always shown too; photo-real try-on is future work	Salma
R4	Privacy of photos and preferences	Low	High	Private encrypted storage; user-scoped queries; AccountDeleted purge; PDPL principles checklist	Karim
R5	Scope creep from many feature ideas	High	Medium	MoSCoW; product-owner approval; stretch list	Youssef
R6	Frontend, backend and AI do not integrate	Medium	Medium	OpenAPI contract first; adapters stubbed from Sprint 1; tests on every pull request	Youssef
R7	Simulated AI in the prototype hides real problems	Medium	Medium	Real adapters built in Sprint 3 behind switches; simulated version kept as fallback	Bashar
R8	Tight capacity (271 of 280 h) and midterms	Medium	Medium	Stretch only after sprint goals; re-estimate every sprint; pair on large stories	Habiba

7 Global, cultural, social, environmental, economic, public health, safety and welfare considerations

Area	How Closet addresses it	Requirements
Global	Local weather for any city; °C; layout ready for other languages	US-04, US-20, NFR-11
Cultural	Modest option respected everywhere; local occasions such as weddings and interviews with default dress codes; Arabic is future work	US-03, US-21, US-07
Social	Closet comments only on clothes, never on bodies or sizes; size is a private field	US-31, NFR-10
Environmental	Utilization and forgotten pieces help people wear what they own; duplicate warning; “donated” is a deletion reason	US-54, US-69, US-65, US-13
Economic	Shopping list suggests only real gaps and replacements; no re-buying “no longer my style”; free to use; AI cost capped	US-32 – US-34, NFR-12
Public health	Weather-appropriate outfits; a note on extreme heat or cold; less daily decision stress	US-20, US-47
Safety	Private by default; export and deletion; learned data visible and undoable	US-06, US-40, NFR-04, NFR-05
Public welfare	Accessible (WCAG 2.1 AA target); honest, explainable advice; limited notifications	NFR-07, NFR-10, NFR-13

8 Traceability

Every use case traces to its committed stories, the components that implement it, and the quality requirements that constrain it. Each committed story’s acceptance criteria form the test suite for that use case.

Use case	Committed stories	Components	NFRs
UC-01 Register / Login	US-01, US-06	API Gateway, Accounts & Profile, Privacy & Settings	NFR-04, NFR-05
UC-02 Manage Style Profile	US-02 – US-05, US-40	Accounts & Profile	NFR-03, NFR-10
UC-03 Add Clothing Item	US-08, US-09, US-11	Wardrobe, Vision, Vision Adapter	NFR-02, NFR-03
UC-04 Edit Clothing Item	US-12, US-71	Wardrobe	NFR-07
UC-05 Delete Clothing Item	US-13	Wardrobe, Outfits, Planner	NFR-05
UC-06 Manage Availability & Laundry	US-15, US-16, US-56	Availability	NFR-03
UC-07 Chat with AI Closet Agent	US-31, US-76	Ask Closet, Language Model Adapter	NFR-01, NFR-06, NFR-12
UC-08 Style This / Forgotten Piece	US-59	Recommendation Engine	NFR-01
UC-09 Get Outfit Recommendation	US-17, 19, 20, 21, 24, 26, 27, 37, 38, 39, 51, 57, 60, 61	Recommendation Engine, Outfits, Weather Adapter, Web App (avatar)	NFR-01, NFR-06, NFR-10, NFR-13
UC-10 Plan Outfit Ahead	US-46, US-47, US-48	Planner, Recommendation Engine	NFR-06
UC-11 Build Outfit	US-42, US-43, US-66	Outfits, Recommendation Engine	NFR-10
UC-12 Mark Outfit as Worn	US-23, US-62	Outfits, Event Bus, Analytics	NFR-03
UC-13 View Wear Analytics	US-52, US-54, US-69	Analytics	NFR-07
UC-14 View Smart Shopping List	US-32, 33, 34, 36, 64, 65	Ask Closet, Wardrobe, Analytics	NFR-12

All 50 committed stories appear exactly once above. NFR-08, NFR-09 and NFR-11 apply to every component.

9 Conclusion

Closet helps people wear more of what they already own. This document defines 70 user stories in 10 epics with acceptance criteria, 14 use cases, and 13 measurable quality requirements, and describes a layered modular monolith in which every module has clear interfaces and every external service has a fallback. We commit to 50 stories, including every Must, by December 1, within 271 of our 280 hours, and we will update this document at every sprint review.